CPSC 481
Artificial Intelligence

Dr. Anand Panangadan
apanangadan@fullerton.edu

What we will cover today

- Uninformed search
  - Depth first search
  - Depth limited search
  - Breadth first search
  - Iterative deepening
- Search algorithm properties

Searching a graph: the challenge

- Number of nodes is practically infinite
  - Cannot pre-compute all nodes and store it in a computer/data structure
  - “Search”, not a “traversal”
- Nodes can reappear in the search
  - Possible to get into loops

Problem to solve

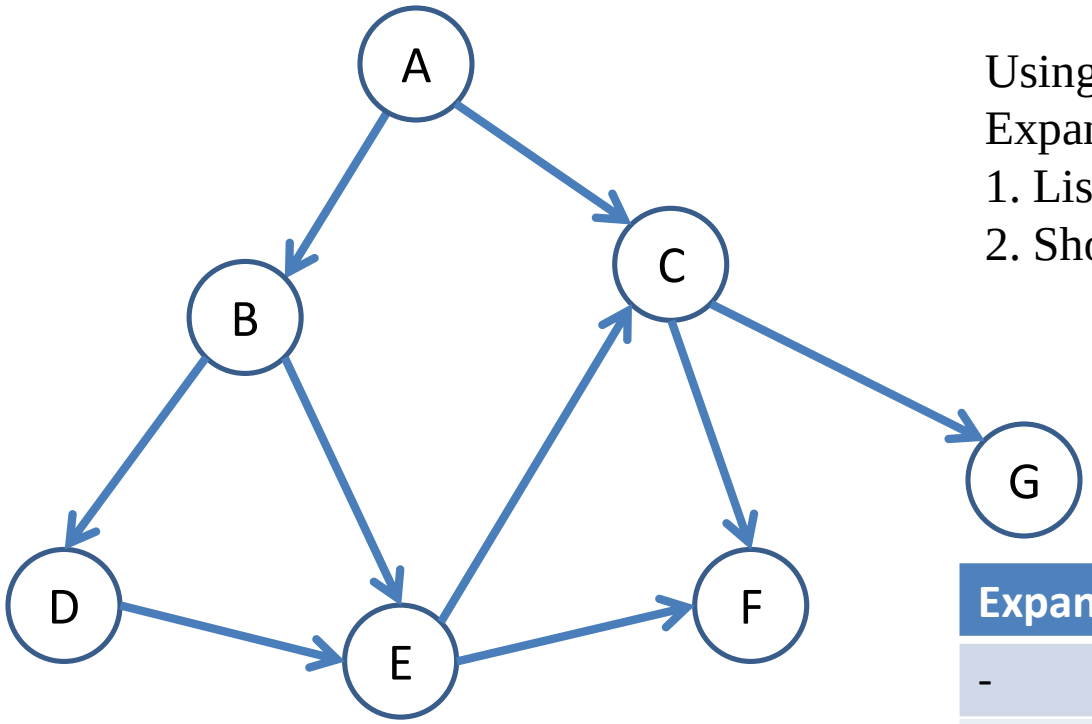
- Represented by a four-tuple [*N*,*A*,*S*,*GD*], where:
- *N:* the problem space, set of vertices
- *A:* the set of edges between vertices
  - the moves in a game ("operators"/ "actions")
- *S:* the start state of the problem
  - Initial board position of the game
- *GD:* the goal state(s) of the problem
  - Checkmate in chess, all numbers sorted in sliding tile puzzles

Depth-first TREE search

- Input: A problem
- Data structure: frontier
  - Also called "open"
  - Stack, LIFO queue

```
Initialize frontier with initial state
Loop do
    IF the frontier is empty RETURN FAILURE
    Choose top node and remove it from frontier
    IF top node is goal RETURN SUCCESS
    expand top node: pushing child nodes to the frontier
```

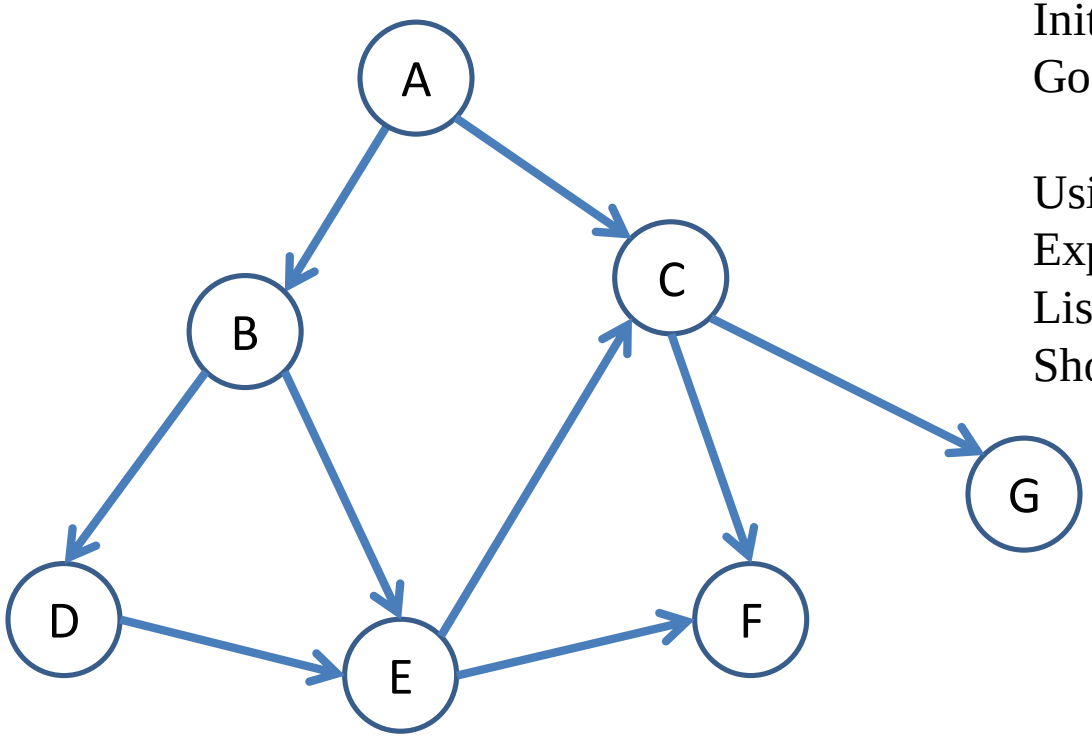
Depth-First Search



Initial state: A
Goal state: D

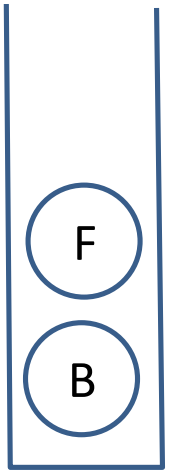
Using DFS (Tree version)
Expand child nodes in alphabetical order
1. List states as they are expanded
2. Show frontier/stack at every step

Expanded node	Frontier
-	A
…	…



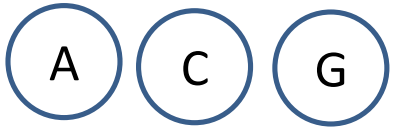
Initial state: A
Goal state: D

Using DFS (Tree version)
Expand child nodes in alphabetical order
List states as they are expanded
Show frontier at every step



Frontier

List of states:

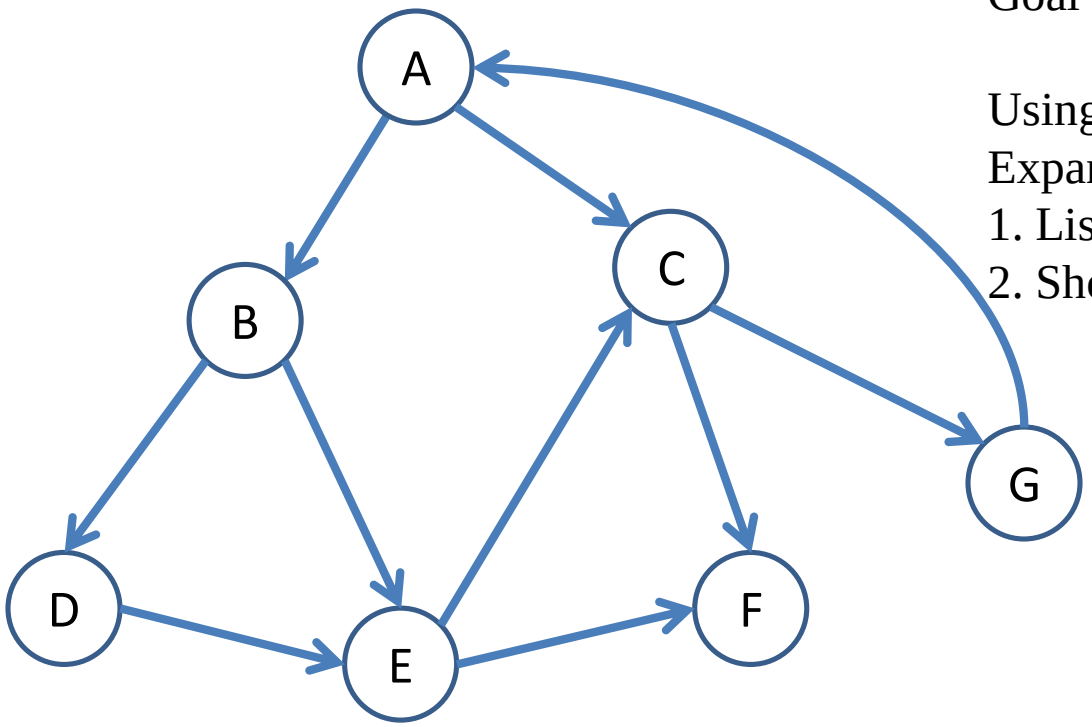


Classwork

- On Google Drive
  - **Link on Canvas**
- Go to today's date and section
  - Editable and viewable by all
- Create a new Google Document and name it with your groupmates' names
  - E.g.: Garcia-Joshi-Nguyen
  - Write the names and role of group members
  - Roles: manager, recorder+presenter, timekeeper
  - Add your answers
- Will not be graded but used for class participation points
  - Can easily share with rest of class outside the breakout room

Depth-first TREE search

- Redundant paths
  - Some states can be reached in more than one way



Initial state: A
Goal state: D

Using DFS (Tree version)
Expand child nodes in alphabetical order
1. List states as they are expanded?
2. Show frontier at every step?

Depth-first TREE search

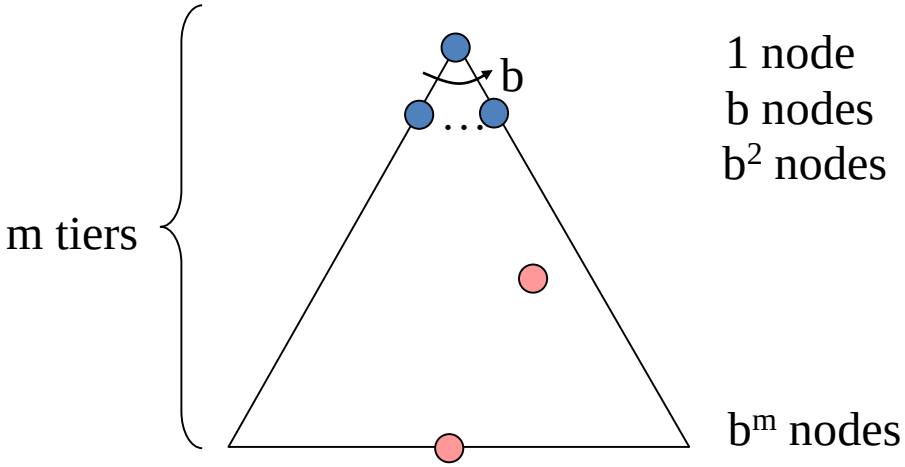
- Redundant paths
  - Some states can be reached in more than one way
  - TREE search can visit the same state multiple times via redundant paths
- Loopy paths
  - A special case of redundant paths

Search Algorithm Properties

- Complete: Guaranteed to find a solution if one exists?
- Optimal: Guaranteed to find the least cost path?
- Time complexity
- Space complexity

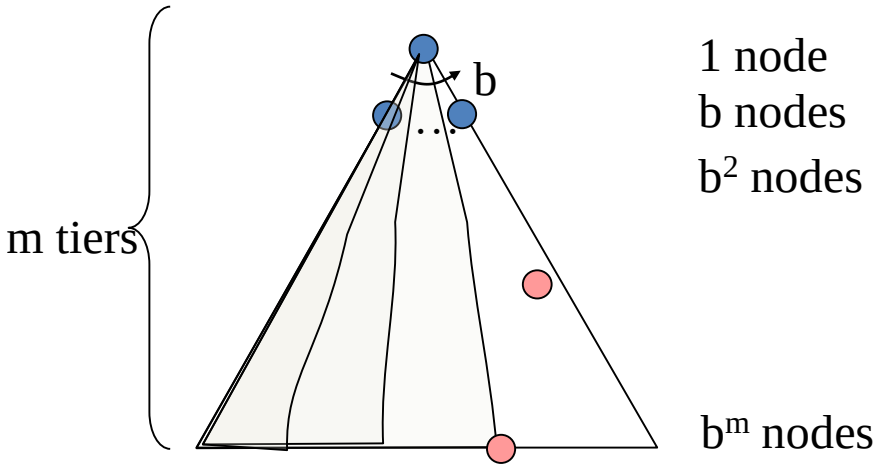
- Cartoon of search tree:
  - **b** is the branching factor
  - **m** is the maximum depth
  - solutions at various depths

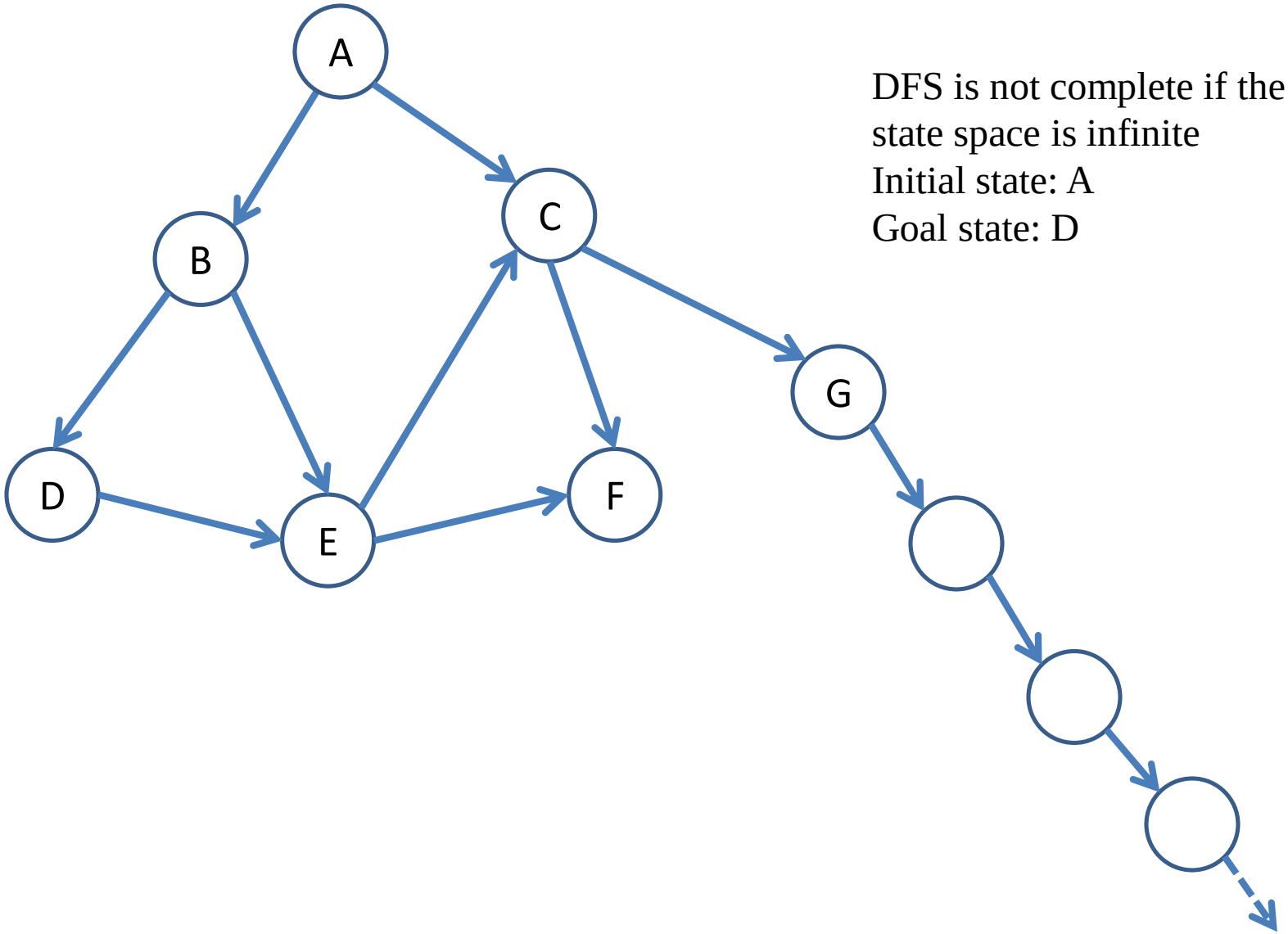
- Number of nodes in entire tree?
  - $1 + b + b^2 + \ldots b^m = b^{m+1} = O(b^m)$



Depth-First Search (DFS) Properties

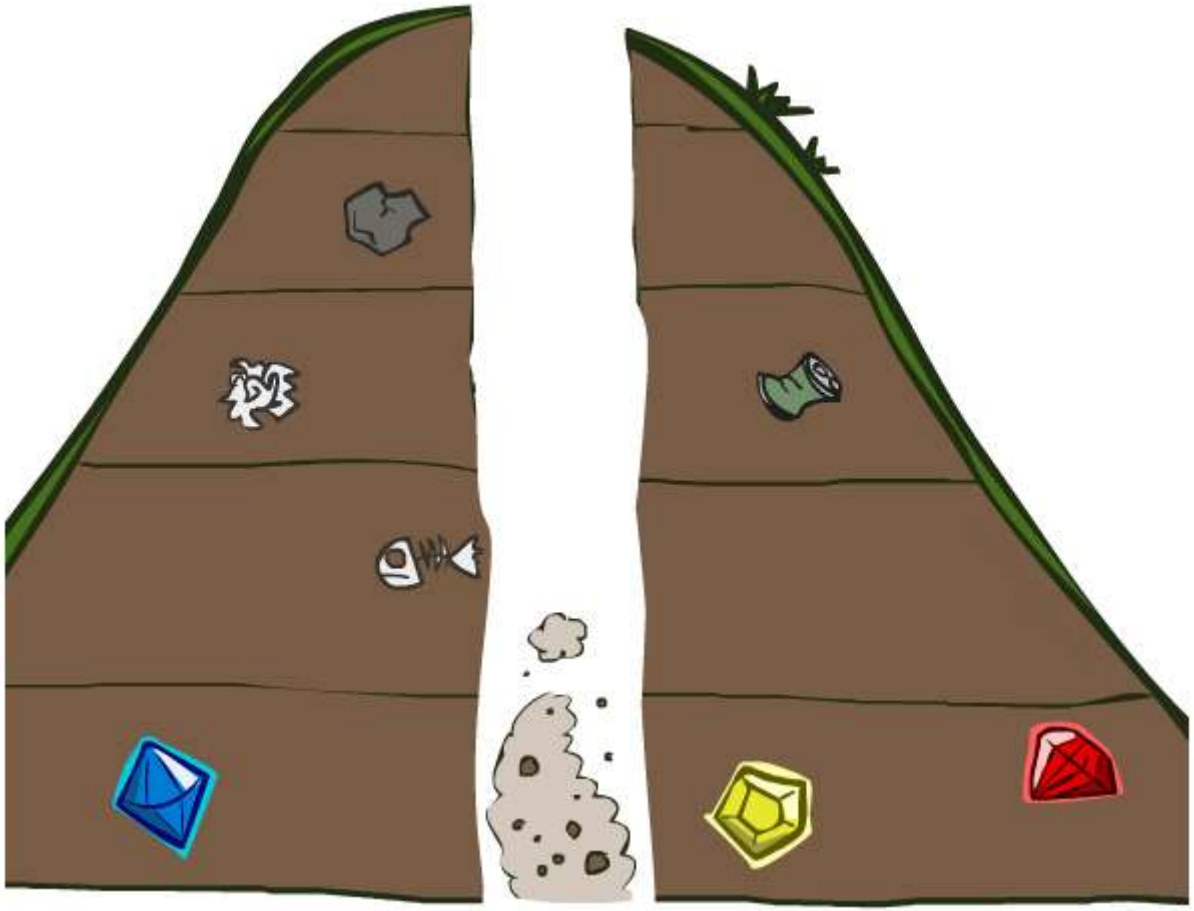
- What nodes does DFS expand?
  - Some left/right prefix of the tree.
  - Could process the whole tree!
  - If m is finite, takes time $O(b^m)$
- How much space does the frontier take?
  - Only has nodes on path to root + siblings for each: so, $O(bm)$
- Is it complete?
  - State space can have loops, or could be infinite, so no
- Is it optimal?
  - No, it finds the "leftmost" solution, regardless of depth or cost





DFS is not complete if the state space is infinite
Initial state: A
Goal state: D

Search Algorithm Properties



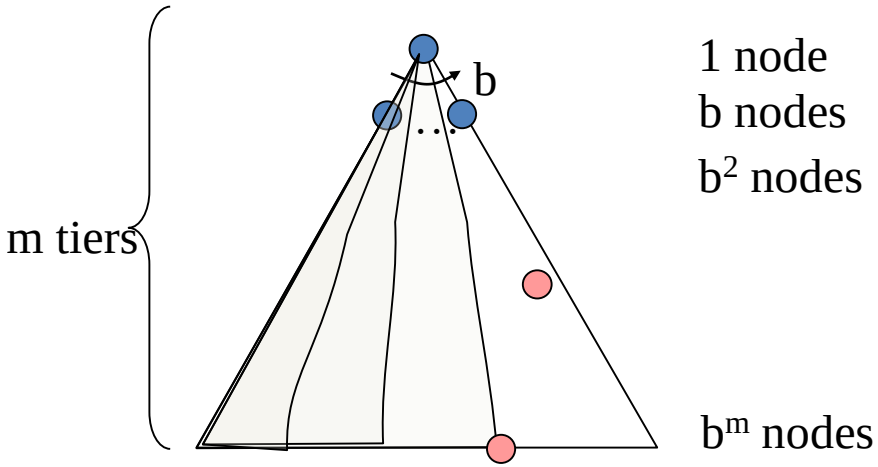
Depth-first TREE search with loop checking

- Input: A problem
- Data structures:
- frontier
  - Stack, LIFO queue
- path from root to current node
  - Parent pointers

```
Initialize frontier with initial state
Loop do
    IF the frontier is empty RETURN FAILURE
    Choose top node and remove it from frontier
    IF top node is goal RETURN SUCCESS
    expand top node
    push resulting nodes to the frontier IF they are not
    already on the path
```

Depth-First Search with loop checking Properties

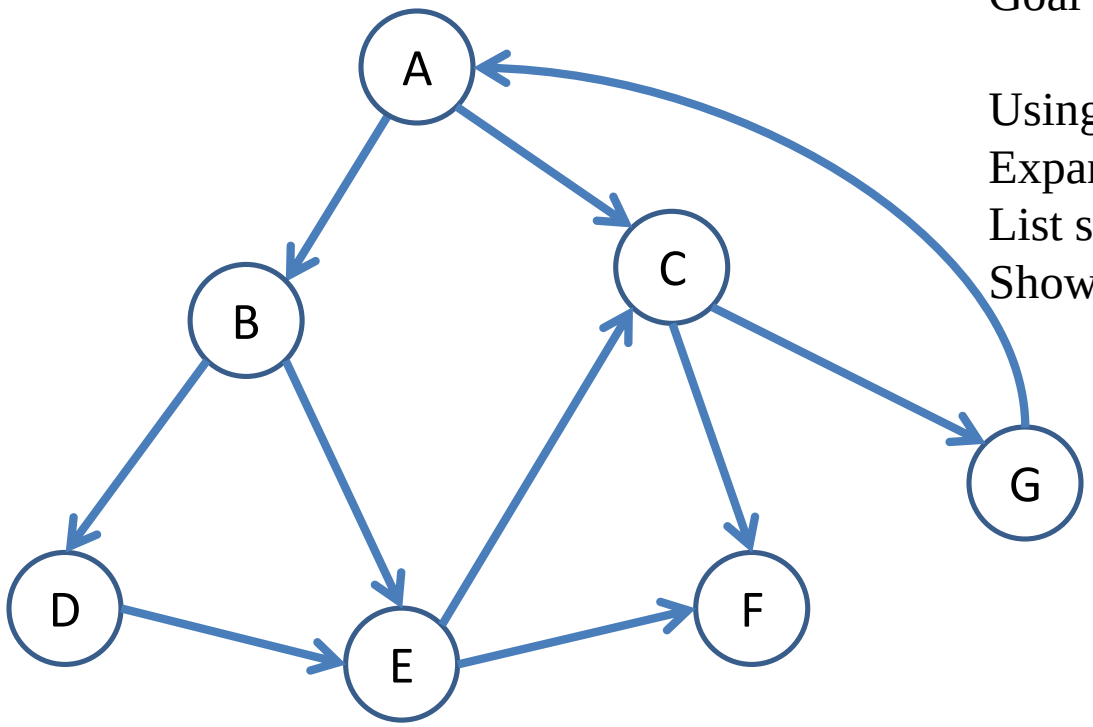
- What nodes does DFS expand?
  - Some left prefix of the tree.
  - Could process the whole tree!
  - If m is finite, takes time $O(b^m)$
- How much space does the frontier take?
  - Only has nodes on path to root + siblings for each, $O(bm)$
  - Path is of length m
  - so, $O(bm)$
- Is it complete?
  - Yes, in finite state spaces
  - But still not in infinite state spaces
- Is it optimal?
  - No, it finds the "leftmost" solution, regardless of depth or cost



Depth-first GRAPH search

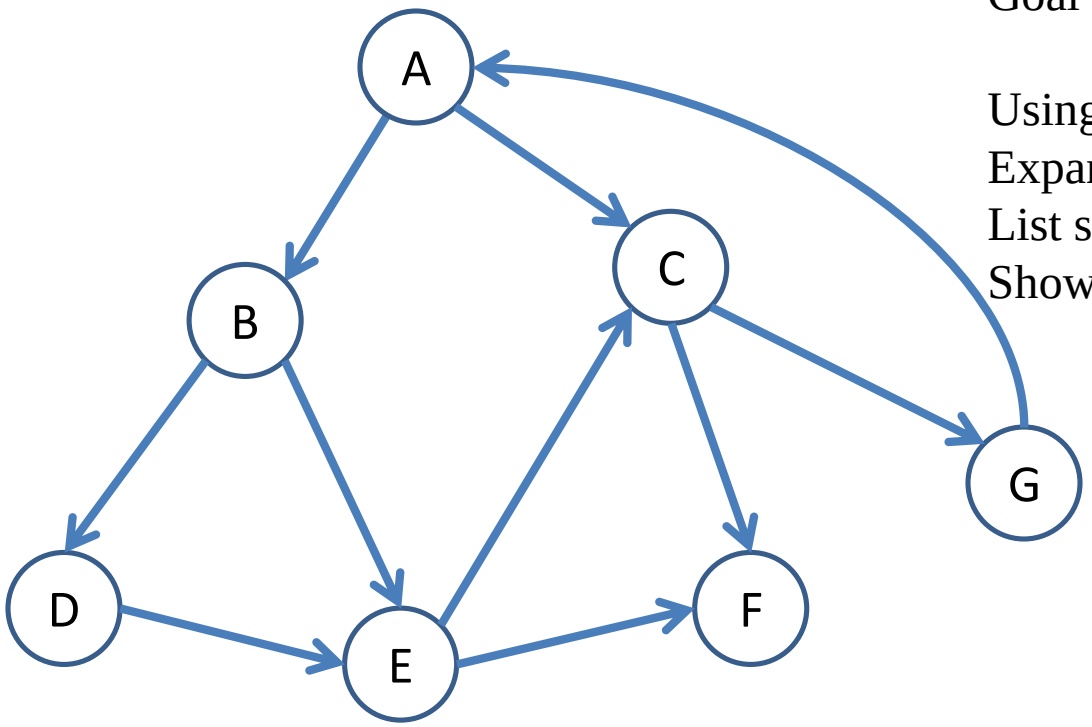
- Input: A problem
- Data structure:
  - Frontier (also called "open")
    - Stack, LIFO queue
  - Explored (also called "closed")
    - Set, for efficiency

```
Initialize frontier with initial state
Initialize explored to empty
Loop do
    IF the frontier is empty RETURN FAILURE
    Choose top node from frontier and remove it
    IF top node is goal RETURN SUCCESS
    Add node to explored
    expand node, pushing resulting nodes to the frontier only if
    not already on frontier or explored
```



Initial state: A
Goal state: D

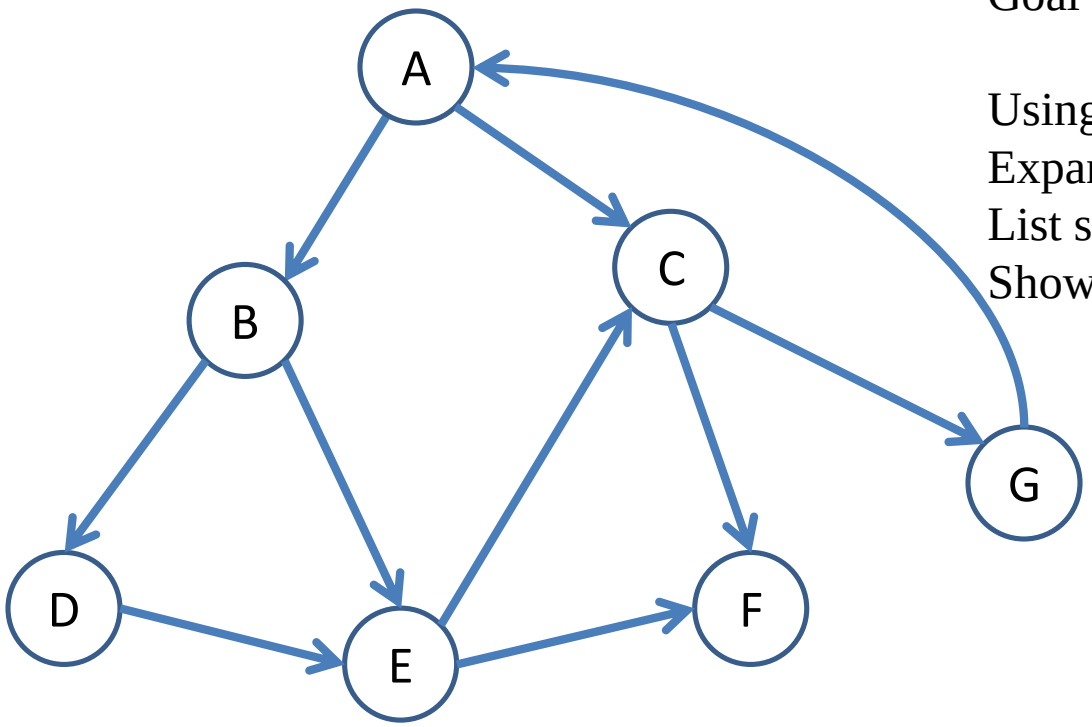
Using DFS (Graph version)
Expand child nodes in alphabetical order
List states as they are expanded?
Show frontier, explored at every step?



Initial state: A
Goal state: D

Using DFS (Graph version)
Expand child nodes in alphabetical order
List states as they are expanded?
Show frontier, explored at every step?

Frontier:[,
Explored: {A, C, G, F, B, E}



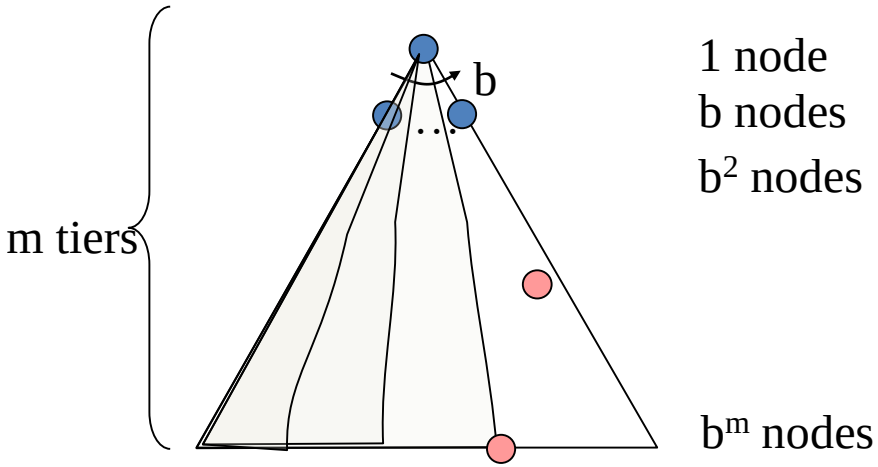
Initial state: A
Goal state: D

Using DFS (Graph version)
Expand child nodes in alphabetical order
List states as they are expanded?
Show frontier, explored at every step?

Frontier:[D
Explored: {A, C, G, F, B , E }

Depth-First Search (Graph version) Properties

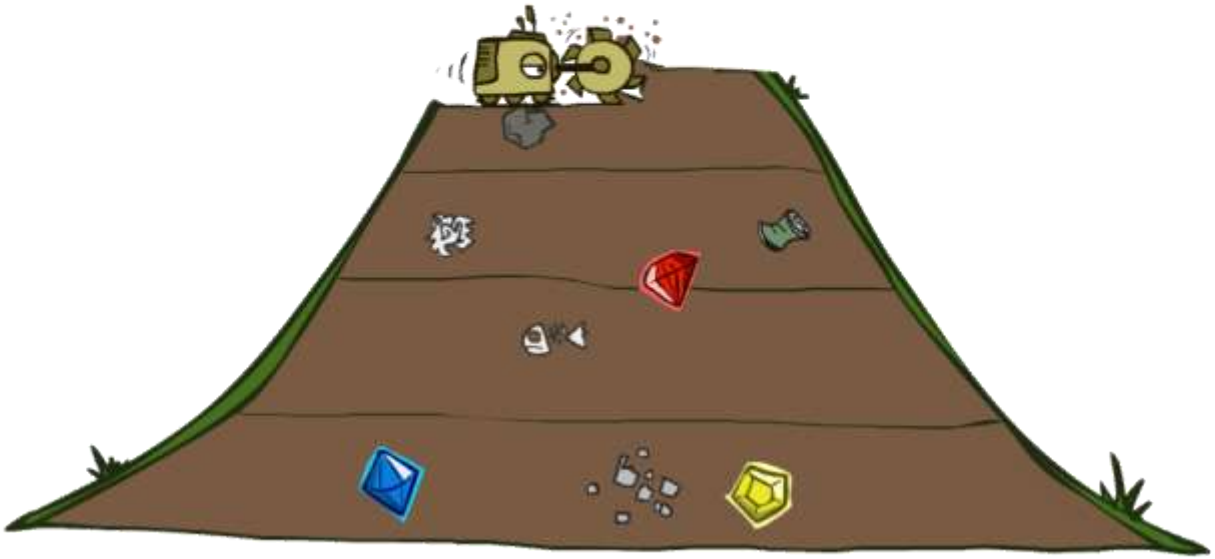
- Time complexity
  - Limited by the state space
  - Can be much smaller than $O(b^m)$
- How much space does the algorithm take?
  - All expanded nodes will be added to explored
  - so, $O(b^m)$
- Is it complete?
  - Yes, in finite state spaces
  - But still not in infinite state spaces
- Is it optimal?
  - No, it finds the "leftmost" solution, regardless of depth or cost



Depth limited DFS

- DFS is not complete for infinite spaces
- How can we adapt DFS for infinite spaces?
- Limit depth to l
- Assumes: maximum depth of solution is known
  - $l \geq d$
- DFS but treat nodes at depth l as if they have no children
- Note:
  - Depth limited DFS fails if we incorrectly set $l < d$
- Time complexity: $O(b^l)$
- Space complexity: $O(bl)$

Breadth-First Search



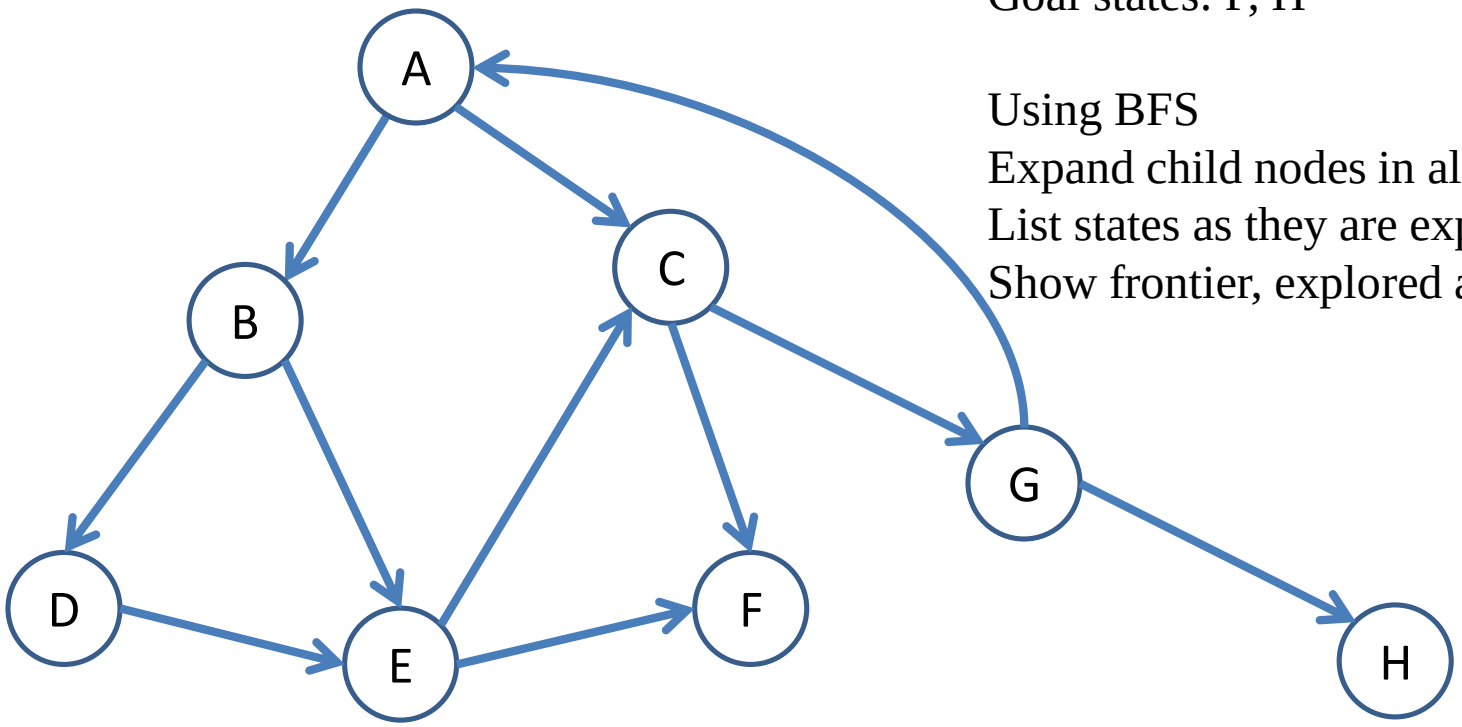
Breadth-first graph search

- Input: A problem
- Data structures:
  - Frontier (also called "open")
    - **Queue, FIFO queue**
  - Explored (also called "closed")
    - Set, for efficiency

```
Initialize frontier with initial state
Initialize explored to empty
Loop do
    IF the frontier is empty RETURN FAILURE
    Choose front-node from frontier and remove it
    Add front-node to explored
    FOR every child-node of front-node
      IF child-node not already on frontier or explored
        IF child-node is goal RETURN SUCCESS
        push child-node to frontier
```

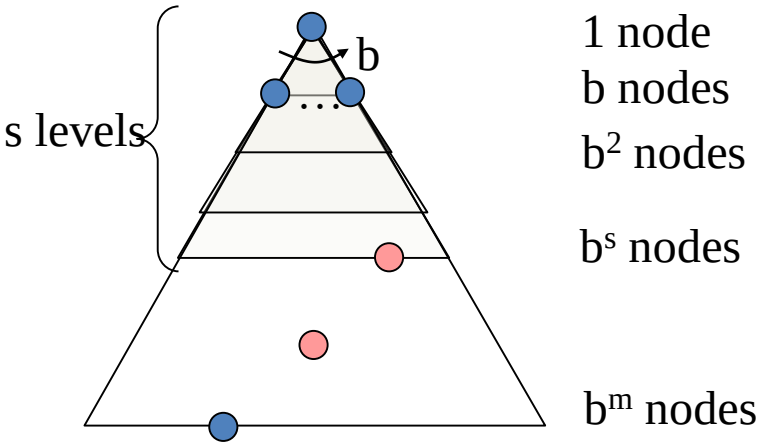
Initial state: A
Goal states: F, H

Using BFS
Expand child nodes in alphabetical order
List states as they are expanded?
Show frontier, explored at every step?



Breadth-First Search (BFS) Properties

- What nodes does BFS expand?
  - Processes all nodes above shallowest solution
  - Let depth of shallowest solution be s
  - Search takes time $O(b^s)$
- How much space does the frontier take?
  - Has roughly the last tier, so $O(b^s)$
- Is it complete?
  - s must be finite if a solution exists, so yes!
- Is it optimal?
  - **Yes**, if costs are all equal
  - more on costs later



DFS vs BFS

Video of Demo Maze Water: BFS

Video of Demo Maze Water: DFS

Time vs Space complexity

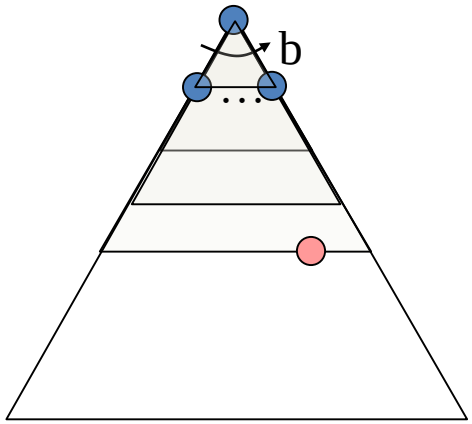
Depth	Nodes	Time	Memory
2	110	0.11ms	107KB
4	11,110	11ms	10.6MB
6	10^6	1.1s	1GB
8	10^8	2 minutes	103GB
10	10^{10}	3 hours	10TB
12	10^{12}	13 days	1PB
14	10^{14}	3.5 years	99PB
16	10^{16}	350 years	10 exabytes

b=10, computer generates 1 million nodes/second, 1KB per node in memory

Space requirement is a much more serious issue than time requirement

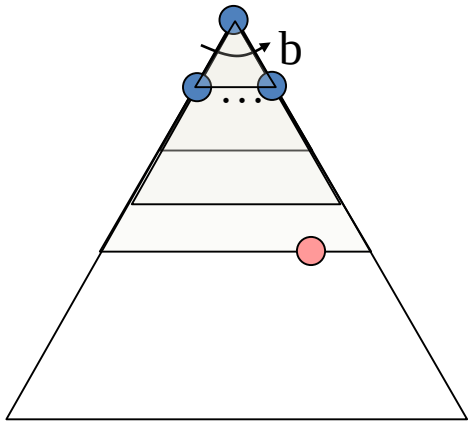
Iterative Deepening

- Idea: get DFS's space advantage with BFS's time / shallow-solution advantages
  - Run a DFS with depth limit 1. If no solution…
  - Run a DFS with depth limit 2. If no solution…
  - Run a DFS with depth limit 3. …..



Complexity of Iterative Deepening

- Isn't that wastefully redundant?
  - Generally most work happens in the lowest level searched, so not so bad!
- Time complexity: $1 + b + b^2 + \cdots b^d = O(b^d)$
- Same as BFS!
- But has the space complexity benefits of DFS



In general, iterative deepening is the preferred uninformed search method when the search space is large and the depth of the solution is not known.

Search Directions in a State Space

- ***Data-driven (or forward)***
  - Use the knowledge and constraints found in each state of the problem to guide search by applying rules/methods to produce new states until it finds a goal state/solution.
    - Most problems can be solved via data-driven approach.
- ***Goal-driven (or backward)***
  - Use knowledge of the goal to guide the search by checking what rules/methods can be used to generate this goal and determine what conditions must be true to use them.
  - These **conditions** become the new goals/subgoals, and continue working backward until it works back to the **facts** of the problem.
    - Diagnosis, theorem proving, answering some multiple-choice questions, etc.
- **Note**: **Both approaches explore the same problem space**.
  - Preferred strategy is chosen by the properties of the problem.
  - **Factors** to consider: **complexity** of **search space**
  - **branching factor**: branching factor can be different near the initial state and at the end states

References

- George Luger, Artificial Intelligence: Structures and Strategies for Complex Problem Solving, 6th edition, Addison Wesley, 2009. **Chapter 3**.
- Russel and Norvig, Artificial Intelligence: A Modern Approach, 3rd edition, Prentice Hall, 2010. **Chapter 3.3**

Acknowledgement

- https://inst.eecs.berkeley.edu/~cs188/su20/